16 • 世界步进与数据生命周期：跟踪一次 Step 到底改了什么

返回总目录

> 前置：第 03、09～12 章。第一次读本章只追踪单线程等价顺序，暂时忽略 SIMD 和原子变量。 目标：沿一个刚体、一个接触、一份冲量缓存，走完当前源码的完整大步。

1. 先建立朴素方案，不先研究调度器

一个最简单的物理世界只需要：

- 刚体数组：位置、旋转、速度、质量、外力。
- 形状数组：圆、盒子等几何。
- 接触数组：哪两个形状、接触点、法线、分离量、累计冲量。
- 可选关节数组。

单线程实现的主干可以用文字写成：

1. 找可能接触的形状对。
2. 更新接触几何和上步冲量缓存。
3. 把外力积分进速度。
4. 施加缓存冲量作为初值。
5. 求解接触和关节。
6. 用求解后的速度积分位置。
7. 保存下次要复用的数据。
8. 对外报告移动与接触变化。

这已经具备物理意义。当前 Box2D 的复杂性主要来自三件事：

- 一个大步被细分为多个子步。
- 数据按照活动状态、颜色和访问频度拆分。
- 几何计算、求解、事件汇总之间存在严格依赖。

接下来把上述朴素顺序逐项映射到源码，而不是把整个 `solver.c` 当作一个必须一次看懂的大函数。

2. 先分清三个数据层

2.1 持久身份：`b2Body`、`b2Contact`

这些对象告诉世界“这是谁、连接谁、现在存在哪里”。

例如一个接触包含两个形状 ID、两端刚体连接边、所属集合、颜色和局部数组位置。它不是接触方程本身。

一个刚体的公开 ID 不因睡眠、唤醒、数组搬迁而改变。数组的局部下标却可能改变。

2.2 持久模拟数据：`b2BodySim`、`b2ContactSim`

- `b2BodySim` 保存质心、变换、力、逆质量和 CCD 所需几何尺度。
- `b2ContactSim` 保存 manifold、材料混合结果、缓存等。

接触从“未 touching”变成“touching”时，其 Sim 数据可能从求解集数组搬进约束图颜色数组。**这只是存放地点变了，不是创建了另一次物理碰撞。**

2.3 热状态与临时约束

`b2BodyState` 是子步中反复改写的数据：

字段	含义	字节数
`linearVelocity`	质心线速度	8
`angularVelocity`	角速度	4
`flags`	动态、锁轴等标记	4
`deltaPosition`	相对大步开始的位移	8
`deltaRotation`	相对大步开始的旋转	8
合计	每个热状态	32

来源：`src/body.h` 第 153–168 行。

Prepare 还会建立临时的接触约束，其中包括有效质量、基础分离量和求解中的累计冲量。它们不是新的世界对象；大步结束后临时内存会释放，必要结果先写回 manifold。

3. `b2World_Step` 的实际顺序

入口：`src/physics_world.c`，`b2World_Step` 第 894–1054 行。

顺序	源码区间	做什么	此时尚未做什么
1	896–929	参数断言、清理若干事件数组、锁定世界、重置任务信息	未推进时间
2	949–954	`b2UpdateBroadPhasePairs`，创建新候选接触	未计算所有精细接触点
3	956–984	建立步进上下文、时长与软度	未施加冲量
4	986–991	`b2Collide` 更新接触及状态迁移	未积分本次大步位置
5	993–999	正时间步时调用 `b2Solve`	传感器最终重叠尚未更新
6	1001–1008	等待剩余树任务	保证后续查询使用可用的树

顺序	源码区间	做什么	此时尚未做什么
7	1010–1015	`b2OverlapSensors`	不因此给物体施加接触冲量
8	1019–1051	临时内存检查、事件缓冲交换、可选记录 hash、解锁	用户之后可读取本步结果

3.1 锁定世界不是什么“大锁保护一切”

`world->locked` 主要阻止用户在内部模拟过程中执行不允许的世界操作。它不是“任何用户回调都可以随便访问世界”的线程安全承诺。

例如 pre-solve 回调可能在并行窄阶段中调用，其用户逻辑必须满足相应线程安全要求。不要在这里创建、销毁刚体，也不要保存内部临时指针供以后使用。

3.2 `timeStep == 0` 不是完全不做事

上下文时间及倒数被置零，跳过 `b2Solve`，但仍执行配对、接触更新、传感器处理和事件缓冲操作。

所以它不是一个纯读取操作，也不等于“任何状态都不变”。由于没有执行 Finalize，本次不会按正步进路径清除外力。

3.3 不要把绘制帧数当成这里的步数

代码只知道调用者给出的物理时间和子步数。固定物理步长累加器、渲染插值属于应用层。第 04 章先讲朴素固定步长方案。

4. 配对阶段：先建记录，不立即产生冲量

入口：`src/broad_phase.c` 的 `b2UpdateBroadPhasePairs`；接触创建见 `src/contact.c` 的 `b2CreateContact` 第 220–369 行。

4.1 一对形状经历的步骤

1. 移动代理查询包围盒树，得到可能相遇的形状。
2. 过滤同刚体、掩码、关节禁止碰撞等不允许的对。
3. 用形状对的 key 排除已有接触。
4. 如果该类型组合有 manifold 函数，创建 Contact。
5. 在 A、B 两个刚体的接触边链中登记它。
6. 初始化缓存和空 manifold，等待窄阶段。

没有对应几何求解函数的组合不会凭空产生接触，例如代码明确指出线段与线段没有该接触实现。

4.2 A/B 次序可能调整

`b2CreateContact` 第 234–238 行会根据注册表的 primary 顺序交换两个形状。

因此调试法线时，先看接触实际保存的 `shapeIdA/B`。不能仅凭你先创建哪个刚体，就认定它永远是 A。

4.3 未 touching 接触放在哪里

- 至少一端醒着：初始进入 awake 集的非 touching 接触数组。
- 两端都不醒着：初始非 touching 接触可以放在 disabled 集。

这里 disabled 集不仅仅是“用户主动禁用刚体的容器”。判断一个接触是否参与求解，要看完整状态，不要只按集合名称猜测。

源码依据：`b2CreateContact` 第 247–258、335–354 行。

5. 窄阶段：计算几何和提交状态变化分开

入口：`b2Collide` 第 687–892 行。

5.1 为什么先收集一张接触指针表

要更新的接触分散在各颜色数组以及 awake 非 touching 数组。当前实现先把它们的地址收集为临时数组，方便统一并行遍历。

此时数组结构保持稳定。每个任务处理自己负责的接触几何，不在任务中任意搬迁整个接触数组。

5.2 每个接触需要计算什么

大体包括：

- 检查 AABB 是否已分离。
- 检查能否采用接触几何回收路径。
- 否则调用具体形状对的 manifold 算法。
- 将局部几何变换为求解器所需锚点和法线。
- 更新材料混合、可选 pre-solve 结果。
- 按接触特征 ID 匹配旧点，复制可用冲量。
- 标记开始、停止 touching 或完全分离。

几何回收不是“重复做一遍 manifold 再热启动”，而是可能跳过重新构造几何，见第 17 章。

5.3 为什么状态变化用 ID 位图记录

因为稍后的数组搬迁会让旧指针失效。每个 worker 记录“哪个 contactId 发生变化”，而不是“哪个地址发生变化”。

任务结束后，主流程对位图按位或，再按确定顺序遍历 contactId。由当前 Contact 的集合、颜色、局部索引重新找到 Sim。

这既避免并行搬迁冲突，也固定了后续修改世界结构的顺序。

5.4 三条状态分支

分支	物理/几何含义	数据操作
`b2_simDisjoint`	包围盒已不再满足保留条件	销毁 Contact
`b2_simStartedTouching`	现在存在可用 manifold 点	连接岛、必要时唤醒、搬进约束图
`b2_simStoppedTouching`	本次没有 manifold 点	断开岛连接、从图搬回非 touching 数组

相应代码在 `b2Collide` 第 815–879 行。

这里 touching 与 `pointCount > 0` 对应，**不必等到分离量变负**。推测接触也可能进入求解图。因此 begin 事件不应被无条件解释为“这一瞬间发生了剧烈撞击”。

5.5 为什么先 Link，再重新取指针，再 AddToGraph

开始 touching 分支的顺序尤其值得阅读：

1. `b2LinkContact` 可能唤醒另一端的睡眠集合。
2. 唤醒可能增长 awake 数组，使原先的 `contactSim` 指针失效。
3. 因此重新从 `awakeSet->contactSims` 取地址。
4. 先复制进约束图。
5. 再从非 touching 数组移除旧 Sim。

这是为了保证“源数据仍然有效”和“索引已经更新”，与冲量公式无关，但缺一步就可能出现内存错误。

6. 时长和软度上下文：只算一次，后续共享

设总步长为 Δt，子步数为 N：

$$h = \Delta t / N, \qquad h^{-1} = N / \Delta t.$$

作用： 给每次子步积分和误差修正使用统一的时间尺度。

例：$\Delta t = 1/60$ s，$N = 4$，得到 $h = 1/240$ s，`inv_h` 为 240 s^{-1}。

本版接触频率实际取：

$$f_c = \min(f_{world}, 0.125/h).$$

变量： f_{world} 是世界设置的接触频率，f_c 是本次真正使用的频率，单位均为 Hz。

作用： 步长较大时，限制求解器试图表达的恢复频率。

代入默认 $f_{world} = 30$ Hz：

子步数	子步长	频率上限	实际 f_c
1	1/60 s	7.5 Hz	7.5 Hz

子步数	子步长	频率上限	实际 f_c
2	1/120 s	15 Hz	15 Hz
4	1/240 s	30 Hz	30 Hz
8	1/480 s	60 Hz	30 Hz

静态接触的软度用两倍该频率构造，并不是再次拿两倍频率与原上限比较。数学推导见第 11 章。

源码：`b2World_Step` 第 977–980 行。

7. 求解器：Prepare 一次，五个阶段重复，Store 一次

入口：`src/solver.c`，`b2Solve` 第 1255–2057 行，核心阶段执行在 `b2SolverTask` 第 1008–1234 行。

7.1 先准备，不先改速度

Prepare 分别处理关节和接触：

- 找醒着的刚体状态索引。
- 拷贝逆质量等反复使用的数据。
- 计算有效质量和基础分离量。
- 根据热启动开关初始化累计冲量。
- 准备静态端 dummy state 所需约定。

这部分每个大步一次。没有必要在每一次约束增量中都重新查对象池、重新计算所有乘积。

7.2 一个子步的输入输出表

阶段	读入	改写	为什么放这里
IntegrateVelocities	当前速度、力、重力、阻尼、h	线速度、角速度	先得到外力驱动的速度
WarmStart	缓存累计冲量、质量、杆臂	两端动态刚体速度	让求解初值接近持续接触的解
Solve / Push	当前速度、分离估计、软度	累计冲量与速度	处理不穿透与位置修正
IntegratePositions	已求解速度、h	大步内累计位移与旋转	把修正速度转化为位置变化
Relax	更新后分离、当前速度、累计冲量	法向、恢复、滚阻、摩擦响应	去掉重叠修正 bias 后再校正速度

注意：Relax 不是只有摩擦，它先重新解法向。漏掉这一项会导致完整算例错误，见第 12 章。

7.3 子步内哪些几何更新，哪些不更新

- 不在每个子步重新运行完整宽阶段和所有 manifold 算法。
- 利用位置增量、旋转增量和初始锚点估计当前分离。

- 冲量计算所用的接触杆臂和有效质量采用准备阶段的数据。
- 更新分离时的旋转锚点，与接触速度 Jacobian 使用固定杆臂，不能混为一个变量策略。

这是“成本、稳定性、几何变化误差”之间的选择，而非精确连续刚体几何的恒等重写。

7.4 Store 的生命周期

子步全部结束后，把接触求解结果写回持久 manifold。下一大步的接触更新可以据此进行 ID 匹配与恢复目标计算。

临时约束缓冲随后被释放。**保存了缓存不等于保存了指向临时缓冲的指针。**

8. 无接触算例：走完一个大步，而不只算最后一条公式

为了只检查时序，设：

- 一个醒着的动态刚体，质量 2 kg。
- 初始世界质心 $c_0 = (5, 10)$ m。
- 初始速度 $v_0 = (1, 0)$ m/s，角速度零。
- 外力 $F = (4, 0)$ N，重力 $g = (0, -10)$ m/s²。
- 无阻尼、无接触、无关节，不触发速度上限。
- 总时间 $\Delta t = 1/60$ s，子步数 2。

8.1 先把力转成加速度

$$a = F/m + g = (4/2, 0) + (0, -10) = (2, -10)\ \mathrm{m/s^2}.$$

作用： 用相同外力和重力更新每个子步的速度。

8.2 每子步更新

无阻尼情况下：

$$v_{k+1} = v_k + ha, \qquad \Delta c_{k+1} = \Delta c_k + h v_{k+1}.$$

变量： Δc 是相对大步开始的累计位移，初始为零；这里使用新速度积分位置。

取 $h = 1/120$ s，则每次速度变化为 $(1/60, -1/12)$ m/s。

时刻	速度 x	速度 y	累计位移 x	累计位移 y
大步开始	1	0	0	0
子步 1 积分速度后	1.016666667	−0.083333333	0	0
子步 1 积分位置后	同上	同上	0.008472222	−0.000694444
子步 2 积分速度后	1.033333333	−0.166666667	0.008472222	−0.000694444
子步 2 积分位置后	同上	同上	0.017083333	−0.002083333

无约束时 WarmStart、Solve、Relax 不增加额外冲量，因此表格没有隐藏省略的接触修正。

8.3 Finalize 才把增量写回持久位置

$$c_{end} = c_0 + \Delta c = (5.017083333, 9.997916667)\ \mathrm{m}.$$

若局部质心为 $(0.2, 0)$ m，且旋转仍为单位旋转，则世界原点为：

$$p_{end} = c_{end} - (0.2, 0) = (4.817083333, 9.997916667)\ \mathrm{m}.$$

作用： 将质心运动转换成用户/形状使用的刚体原点变换。不能把质心位置直接当成 `transform.p`。

8.4 外力为什么不能在第一个子步后清掉

如果第一个子步后清力，第二个子步将失去水平 2 m/s² 的加速度，只推进了一半的施力时间。

本版在 Finalize 清除 `force`、`torque`，使本大步内各子步都能读取同一份外力累计。下一大步若还需持续施力，应由应用按其输入策略继续提供。

源码：`b2FinalizeBodiesTask` 第 631–633 行。

9. Finalize：不是只做一次位置加法

`b2FinalizeBodiesTask` 第 560–750 行承担多个收尾工作。

9.1 先写位姿，再计算状态指标

1. 持久质心加累计位移。
2. 大步旋转与累计旋转相乘并归一化。
3. 计算线速度与转动影响下的运动尺度。
4. 用真实速度和位置修正量共同评估睡眠。
5. 清空增量，反推出刚体原点。
6. 填移动事件。
7. 清外力和力矩。
8. 判断 fast、调度普通 CCD 或登记 bullet。
9. 更新形状包围盒、标记需要更新的树代理。

这里的“写回位置”可能随后被 CCD 修正。因此只在 Finalize 最前面打断点，不一定看到最终对外位置。

9.2 为什么睡眠还看位置变化

Relax 可能把修正速度去掉，但物体这一步仍然被位置修正移动了不少。如果只看最终速度，可能把仍在修正的堆叠错误地判为稳定。

具体速度指标推导和数字例在第 18 章。

9.3 普通 fast 与 bullet 分开

- 普通高速动态刚体在 Finalize 中调用连续求解，查询静态树。
- 高速 bullet 先登记，等相关树状态更新后统一处理，还会查询运动学和动态树。
- 目标为 bullet 的候选被当前连续路径跳过。

这些限制来自本版执行代码，不是说“所有运动物体两两都有完整 CCD”。详见第 08 章。

10. `b2Solve` 后半段为何按这个次序收尾

顺序	源码区间	原因
1	1583–1610	等岛拆分完成后 Finalize，避免同时改岛结构
2	1624–1795	收集关节事件和 hit 事件，使用已保存的冲量结果
3	1798–1890	等树任务、汇总需要扩大的 AABB，建立确定顺序的移动代理记录
4	1893–1960	处理高速 bullet 并扩大相关代理
5	1967–1998	汇总连续路径发现的 sensor hits
6	2000–2055	最后把符合条件的岛移到睡眠集合

为什么不能先睡眠再处理树更新

睡眠会把 awake 稠密数组中的对象搬走，并可能用末尾元素补洞。前面按 bodySimIndex 记录的 enlarged 位图就可能指向另一个刚体。

因此代码明确把岛睡眠放到最后。这是一条数据依赖，不是可随意调换的性能偏好。

11. 接触跨大步：四种数据不要混淆

数据	表示什么	产生与使用
`normalImpulse`	当前接触点保留的累计法向冲量	Store 写回，下一次作为热启动缓存
`tangentImpulse`	累计切向冲量	同上，用于持续摩擦
`totalNormalImpulse`	本大步各子步实际施加的法向冲量的代数累计	包含 WarmStart 与 Solve/Relax 的有符号增量，不通常等于最终 `normalImpulse`
`normalVelocity`	本步窄阶段、积分速度和热启动之前采样的法向相对速度	直接写入持久 manifold，供下一步恢复和当前 hit 使用；不是求解结束速度

`totalNormalImpulse` 在 Prepare 清零，在 WarmStart 加缓存冲量，在 Solve/Relax 加投影后的增量，包括负增量。它不是仅正增量之和，也不是历史最大冲量。对应 `contact_solver.c` 第 114、190、309–315 行。若要换算本大步的等效平均法向力，应除以大步时长，而不是子步时长。

`normalVelocity` 的正常更新位于 `b2CollideTask` 第 621–638 行，几何回收路径更新位于第 560–576 行。Store 写回冲量，不写回这个速度。

`b2UpdateContact` 第 603–650 行先匹配特征 ID；匹配成功时复制缓存。若旧点的 `totalNormalImpulse > 0`，且旧的接近速度超过阈值，则生成恢复目标；“中途曾经施加正增量”本身不足以代替这个最终判据。

一个只检查字段生命周期的例子

设旧点记录：

- 旧法向速度 −4 m/s。
- 旧 `totalNormalImpulse` 大于零。
- 新旧特征 ID 相同。
- 混合恢复系数 0.5，阈值 1 m/s。

本步接触更新得到恢复速度：

$$u_r = -e v_n^{old} = -0.5 \times (-4) = 2\ \mathrm{m/s}.$$

作用： 存一个正的目标分离速度。Prepare 只把它复制进临时约束，Relax 再把它转为法向速度目标。

若写为速度方程中的 bias，则为 −2 m/s，因为目标方程是“当前法向速度减去 2 等于零”。不能把正的 `restitutionVelocity` 直接当成正的 `velocityBias`。

这个例子只说明字段和符号，不能保证任意复杂接触最终恰好弹回 2 m/s；还受投影、其他点和恢复清除分支影响。

12. 事件不是一个统一的“碰撞回调”

12.1 三种接触信号分别意味着什么

- begin：从没有可用接触点变成有点，可能是推测接触。
- end：停止该接触关系或接触被销毁等。
- hit：接近速度超过阈值，并且该点整步 `totalNormalImpulse > 0`；不是只要中途出现过一个正增量就一定触发。

本版 hit 汇总在 `b2Solve` 第 1734–1746 行检查接近速度和 `totalNormalImpulse`。仅凭 AABB 重叠，不会自动得到 hit。

两个点都满足条件时，当前代码选择更大接近速度的点作为该接触的事件代表，而不是对每个点无条件发一个事件。

12.2 传感器为什么放到位姿求解之后

传感器不需要施加阻止运动的冲量，主要回答“现在有哪些 visitor 在里面”。放在求解和 CCD 收尾之后，可以按更新后的世界状态计算重叠，并融合相应连续路径记录。

入口：`src/sensor.c`，`b2OverlapSensors` 第 239–369 行。

12.3 新旧集合怎样生成 begin/end

例：旧 visitor 集合为 `{2,5}`，新集合为 `{5,9}`，且相同 ID 的 generation 没变化。

- 旧有新无：2，生成 end。
- 新有旧无：9，生成 begin。
- 新旧都有：5，保持持续重叠，不重复生成 begin。

实际实现比较已整理的 visitor 列表，同时比较 generation。若一个槽位销毁后被新形状复用，不能把相同 shapeId 当成同一个 visitor。

12.4 双缓冲的“previous”不等于上一物理帧

假设当前写入 end 事件的数组索引是 0：

1. 本次步进把结束事件写入数组 0。
2. Step 结尾把写入索引切到 1，并清数组 1。
3. 读取接口使用 `1 - world->endEventArrayIndex`，因此读到数组 0，也就是**刚完成这次步进的事件。**

不要把源码里的“previous buffer”译成“用户总是读到上一帧事件”。它指交换后的另一侧缓冲。

读取函数位于 `physics_world.c`：`b2World_GetSensorEvents` 第 1502–1523 行，`b2World_GetContactEvents` 第 1526–1551 行。

数组内容由世界管理。应按 API 生命周期及时消费，需要跨步保留时复制业务所需数据，并检查对象句柄是否仍有效。

13. 到这里才看优化：同一物理工作为何出现 Stage、Block、Color

13.1 Stage 是阶段，不是一条物理约束

Prepare、积分速度、WarmStart 等各是一种工作阶段。某些阶段按颜色拆成多个 stage 条目。

本版正常求解每个子步有一次 Solve 和一次 Relax。若活跃颜色数为 C，阶段表条目数为：

$$S = 5 + 3C.$$

推导： 两个 Prepare、一个积分速度、一个积分位置、一个 Store，共 5；WarmStart、Solve、Relax 各有 C 个颜色条目，共 $3C$。

代入： 3 个活跃颜色时，阶段表有 14 个条目。

这张表被子步循环重用，不为每个子步重新分配一份。

若有 N 个子步，阶段表执行次数为：

$$E = 3 + N(2 + 3C).$$

解释： 两个 Prepare 加 Store 只执行一次；每子步两个积分阶段，加三类颜色阶段。

代入： $C = 3, N = 4$ 时是 $3 + 4 \times 11 = 47$ 次。这里统计阶段表执行，不包含单独调用的 overflow 工作，也不是物理约束数量。

代码依据：`b2Solve` 第 1428–1445 行，`b2SolverTask` 第 1058–1169 行。

13.2 Block 是分工单位

一个阶段可以包含多个工作块，由 worker 领取。两个工作块能否同时执行，取决于它们有没有写同一份动态刚体状态。

颜色解决这种依赖；不是因为“多线程写同一个 float 也许没问题”。细节见第 19 章。

13.3 为什么 Prepare/Store 可以跨颜色铺平

它们主要处理各自约束数据，而不是像 Solve 那样同时修改两端动态刚体速度，所以可采用统一的平铺区间，减少小颜色带来的调度碎片。

源码第 1350–1354 行明确说明该设计。不能据此推导“所有阶段都可以去掉颜色”。

13.4 源码注释也可能落后于实现

`b2SolverTask` 第 1040–1050 行的阶段序列注释还提到 `b2_stageRestitution`。但 `src/solver.h` 第 75–85 行枚举和实际执行路径没有独立恢复阶段。

本书采用当前执行代码：恢复包含在 Relax 接触求解中。

14. 调试时应在哪些点观察什么

断点	看什么	不应期待什么
`b2CreateContact` 结束	对象关系、空 manifold、缓存初值	不期待立刻有冲量
`b2UpdateContact` 结束	法线、锚点、分离、ID、恢复来源	不期待本步已改速度
Prepare 结束	`normalMass`、`baseSeparation`、缓存累计值	不期待热启动已经施加
WarmStart 结束	刚体速度是否收到缓存响应	不期待所有约束已满足
Solve 结束	带修正目标的速度和累计冲量	不期待摩擦一定已更新
积分位置结束	`deltaPosition`、`deltaRotation`	不期待持久 `transform` 已写回
Relax 结束	法向、恢复、摩擦之后的残差	不假设一遍解完所有耦合
Store 结束	manifold 的冲量和事件所需字段	不保留临时约束指针
Step 返回	最终位姿、事件、睡眠与 CCD 标志	不继续使用上一阶段内部地址

15. 练习与答案

题 1：接触刚变成 touching，为什么还要重新读取 `contactSim`？

连接接触可能唤醒睡眠物体，使 awake 数组扩容。旧地址可能已失效，必须按当前索引重新获取。

题 2：本步没有移动的刚体一定没有参与求解吗？

不一定。它可能一直醒着，通过接触冲量平衡重力，最终位移很小。是否参与由求解集合和约束状态决定，不由画面是否移动决定。

题 3：缓存冲量、恢复速度都在 Prepare 里重新从零算吗？

不是。冲量来自持久 manifold 的缓存；恢复目标在接触更新或回收路径中利用旧步数据生成，Prepare 主要搬运并计算本步求解准备量。

题 4：为何不能在睡眠迁移以后继续使用旧 enlarged 位图？

它按原 awake Sim 下标编码；RemoveSwap 搬迁之后，同一位可能对应另一刚体。

题 5：碰撞 begin 一定意味着声音系统应该播放撞击音吗？

不一定。begin 可以来自推测接触。需要撞击强度时，应考虑 hit 事件及其阈值和业务条件。

继续：17 · 宽阶段动态树与几何缓存 · 返回总目录